CampusFlow Inc.

Campus Flow

Software Requirements Specification

Version <2.0>

Revision History

Date	Version	Description	Author
20 / APRIL / 2026		Phase 2: Design Report	Kapil Tamang, Ibsan Hossain, Tausif Hassan, Arif Hossain

CampusFlow Inc.	Version: 2.0
Phase 2	Date: 16/NOV/2025
AI Enabled Online College Program Management System	

Table of Contents

1. Introduction

1.1. Overview of the System

1.2. Collaboration Class Diagram

2. Use Cases

2.1. List of Use Cases

2.2. Scenarios for Each Use Case

2.3. Petri Net

3. Entity-Relationship (E-R) Diagram

3.1. Complete E-R Diagram of the System

3.2. Attributes and Keys for Each Class

4. Detailed Design (Pseudo-Code for Every Method)

5. System Screens

5.1. Major GUI Screens

5.2. Sample Prototype of Selected Functionality

6. Team Work Memos

7. Git Repository

CampusFlow Inc.	Version: 2.0
Phase 2	Date: 20/April/2026
AI Enabled Online College Program Management System	

Design Report

1. Introduction

1.1 Overview of the System:

CampusFlow, which is the AI-based online college program management system, is an online platform that serves to centralize key academic and administrative activities associated with the college program. There are four main types of users of the application who include Visitor, Student, Instructor, and Registrar. Users have the ability to view the information provided through the platform, apply to become a student or instructor, enroll in different classes, grade students' work, conduct review activities, make complaints, apply for graduation, among other tasks.

Tech Stack:
Frontend: Streamlit
Backend: Python-based application logic
Database: Centralized data storage for users, courses, semesters, enrollments, grades, reviews, complaints, applications, and chatbot sessions

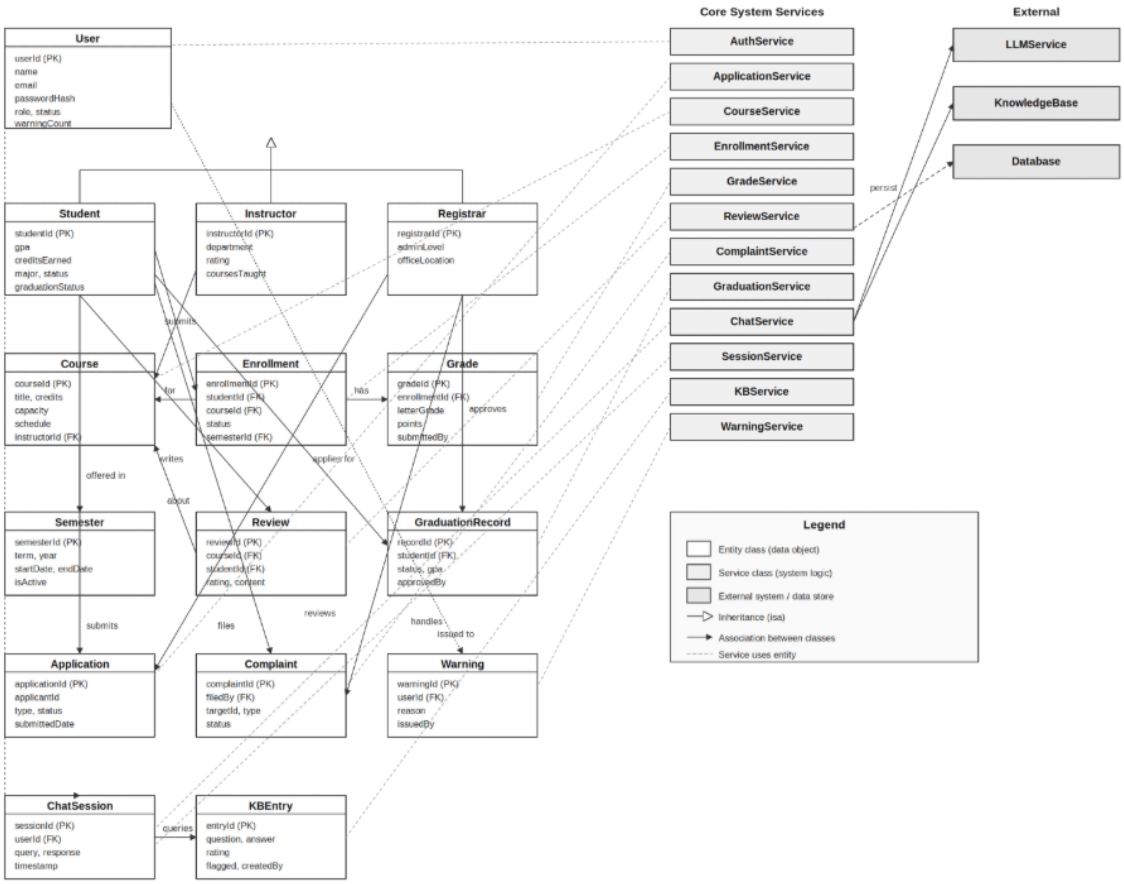
Core System Services and Features:

- **AuthService:** Responsible for handling authentication processes such as login, logout, and registration for visitors, students, instructors, and registrars.
- **ApplicationService:** Manages application services for students and instructors and supports registrar workflows for application approval and rejection.
- **CourseService:** Manages courses, including creating, browsing, assigning, scheduling, and managing course availability.
- **EnrollmentService:** Manages course enrollment by allowing students to register for a course, drop a course, check for course conflicts, and handle waitlists.
- **GradeService:** Supports grade submission for instructors and allows students to view their academic information and GPA.

- **ReviewService:** Allows users to rate courses and write reviews, while also filtering inappropriate submissions.
- **ComplaintService:** Handles complaint submission and supports warning-related actions.
- **GraduationService:** Supports graduation application processes for students and registrars.
- **ChatService:** Offers chatbot service powered by an AI model using the local knowledge base with an alternative LLM if necessary.
- **SessionService:** Stores user queries, chatbot responses, and timestamps in chat sessions.

Our prototype uses Streamlit for the front end along with Python-based application logic and a centralized database that manages users, courses, semesters, enrollments, grades, reviews, complaints, applications, and chat sessions.

1.2 Collaboration Class Diagram


Core System Services
External
User
userId (PK)
name
email
passwordHash
role, status
warningCount
Student
studentId (PK)
gpa
creditsEarned
major, status
graduationStatus
Instructor
instructorId (PK)
department
rating
coursesTaught
Registrar
registrarId (PK)
adminLevel
officeLocation
Course
courseId (PK)
title, credits
capacity
schedule
instructorId (FK)
Enrollment
enrollmentId (PK)
studentId (FK)
courseId (FK)
status
semesterId (FK)
Grade
gradeId (PK)
enrollmentId (FK)
letterGrade
points
submittedBy
Semester
semesterId (PK)
term, year
startDate, endDate
isActive
Review
reviewId (PK)
courseId (FK)
studentId (FK)
rating, content
GraduationRecord
recordId (PK)
studentId (FK)
status, gpa
approvedBy
Application
applicationId (PK)
applicantId
type, status
submittedDate
Complaint
complaintId (PK)
filedBy (FK)
targetId, type
status
Warning
warningId (PK)
userId (FK)
reason
issuedBy
ChatSession
sessionId (PK)
userId (FK)
query, response
timestamp
KBEntry
entryId (PK)
question, answer
rating
flagged, createdBy
AuthService
ApplicationService
CourseService
EnrollmentService
GradeService
ReviewService
ComplaintService
GraduationService
ChatService
SessionService
KBService
WarningService
LLMService
KnowledgeBase
Database
persist
submits
for
has
approves
writes
applies for
offered in
about
submits
files
reviews
handles
issued to
queries
Legend
Entity class (data object)
Service class (system logic)
External system / data store
Inheritance (isa)
Association between classes
Service uses entity

2. Use Cases

2.1 List of Use Cases

The CampusFlow system supports the following use cases, grouped by user role. Each use case is detailed in section 2.2 with both a normal scenario and exceptional scenarios.

- User Authentication: Students, Instructors, Registrars, and Visitors can register, log in, or log out. The system verifies credentials and blocks access for closed or blacklisted accounts.
- Apply to the Institution: Visitors can apply to become students or instructors. Registrars then review, approve, or reject these requests
- Browse Courses: Students, Instructors, and Visitors can search the course catalog. They can filter by details like credits, instructor, and schedule.
- Course Enrollment: Students can add, drop, or swap courses. The system checks for prerequisites and schedule conflicts, using a waitlist for full classes.
- Submit and View Grades: Instructors enter grades for their classes. Students can view these grades and track their GPA.
- Course Reviews: Students can rate and review courses. A filter removes any inappropriate content.
- Complaint and Warning Management: Users can file complaints. Registrars resolve these issues and may issue warnings that lead to account suspension.
- Apply for Graduation: Students apply for graduation once eligible. Registrars verify all requirements before giving final approval
- Manage Courses and Semesters: Registrars create semesters, schedule courses, and assign instructors.
- AI Chatbot Interaction: All users can ask the chatbot questions. It uses a local knowledge base (KB) or an external LLM to provide answers. Users can rate KB answers to help improve them.

2.2 Scenarios for Each Use Case

User Authentication

Normal Scenario:

- A visitor opens CampusFlow and selects Register or Login.

- For registration, the visitor enters credentials; the system checks uniqueness and that the email is not on the blacklist.
- For login, the system validates the email and password, loads the role (Student, Instructor, or Registrar), and grants role-appropriate access.
- On success, the user is redirected to their role-specific dashboard.
- Logout clears the session and returns the user to the landing page.

Exceptional Scenarios:

- Duplicate email or username
  - Condition: Email or username already exists in the database.
  - System response: Registration is rejected; displays "Account already exists with this email."
- Blacklisted user attempts registration
  - Condition: Email or identifier is on the blacklist.
  - System response: Registration is blocked; displays "You are not allowed to register with this institution."
- Invalid credentials on login
  - Condition: Email not found or password incorrect.
  - System response: Displays "Invalid email or password" and keeps the user as a Visitor.
- Closed or suspended account
  - Condition: User account status is CLOSED or SUSPENDED.
  - System response: Login is rejected; displays a status-specific message (e.g. "This account has been suspended.").

Apply to the Institution

Normal Scenario:

- A visitor opens the application form and selects Student or Instructor.
- The visitor fills in personal information, academic background (for students), or credentials and department (for instructors), then submits.

- The system stores the application with status PENDING and notifies the registrar dashboard.
- The registrar reviews the application and accepts or rejects it.
- On acceptance, the account is created and the applicant receives login access with the appropriate role.

Exceptional Scenarios:

- Incomplete application
  - Condition: One or more required fields are missing.
  - System response: Submission is blocked; the form highlights missing fields.
- Duplicate application
  - Condition: The same applicant already has a PENDING application of the same type.
  - System response: Rejects the duplicate and shows "You already have a pending application."
- Application rejected by registrar
  - Condition: Registrar marks the application as REJECTED with a reason.
  - System response: Applicant is notified with the rejection reason and may reapply after an institution-defined waiting period.

Browse Courses

Normal Scenario:

- A user navigates to the course catalog.
- The system loads all courses for the active semester with title, credits, instructor, schedule, and remaining capacity.
- The user filters by department, credit hours, or instructor and searches by keyword.
- The user selects a course to see a detailed view, including prerequisites, description, and past reviews.

Exceptional Scenarios:

- Catalog unavailable
  - Condition: Course data cannot be loaded (database error).
  - System response: Displays "Catalog temporarily unavailable" and hides filter controls.
- No courses for semester
  - Condition: No courses exist for the active semester.

    - System response: Displays "No courses have been scheduled for this semester."
- Search returns no results
    - Condition: Filter and search criteria match zero courses.
    - System response: Shows "No matching courses" and offers to clear filters.

Course Enrollment

Normal Scenario:

- A student browses the catalog and selects a course.
- The system checks that the student has met the prerequisites and has no schedule conflict.
- The system checks seat availability: if a seat is open, the student is enrolled; otherwise the student is offered a waitlist slot.
- On successful enrollment, an Enrollment record is created with status ENROLLED, the course capacity is decremented, and the student's schedule is updated.
- Students may drop a course before the drop deadline, which increments the course capacity and promotes the next waitlisted student.

Exceptional Scenarios:

- Prerequisite not met
    - Condition: Student is missing a required prerequisite course.
    - System response: Enrollment is rejected; displays the missing prerequisite and blocks registration.
- Schedule conflict
    - Condition: Course meets at the same time as another enrolled course.
    - System response: Enrollment is rejected; displays the conflicting course and prevents registration.
- Course full
    - Condition: No open seats in the course.
    - System response: Offers the student a place on the waitlist; student is automatically promoted when a seat opens.
- Drop deadline passed
    - Condition: Student tries to drop after the institutional drop deadline.
    - System response: Drop is rejected; a notice is shown and the student must petition the registrar.

Submit and View Grades

Normal Scenario:

- An instructor opens the grade submission page for a section they teach.
- The system loads the roster and any previously entered grades.
- The instructor enters or edits grades and submits; the system validates grade values and marks them FINAL after the deadline.
- Students see their grades on their transcript view and the GPA is recalculated automatically.

Exceptional Scenarios:

- Unauthorized grade entry
  - Condition: An instructor tries to enter grades for a section they do not teach.
  - System response: Access is denied with "You are not authorized to grade this section."
- Invalid grade value
  - Condition: A submitted grade is outside the allowed set (e.g. not A–F or a permitted special grade).
  - System response: The specific entry is rejected and the instructor is prompted to correct it.
- Grade edit after finalization
  - Condition: Grades have already been marked FINAL.
  - System response: Edit is blocked; instructor must submit a grade-change request to the registrar.

Course Reviews

Normal Scenario:

- A student who has completed a course opens the review form for that course.
- The student provides a star rating (1–5) and a written review.
- The system runs a content filter; if the review passes, it is stored and shown anonymously on the course page.
- The aggregate course rating is recalculated and displayed on the course listing.

Exceptional Scenarios:

- Review submitted by non-completer
  - Condition: User has no completed enrollment for the course.

    - System response: Submission is rejected; displays "Only students who have completed this course can review it."
- Inappropriate content detected
    - Condition: Content filter flags profanity or personal attacks.
    - System response: Review is hidden pending registrar review; student is notified.
- Duplicate review
    - Condition: Student has already submitted a review for this course.
    - System response: Existing review is loaded into the form for editing rather than creating a duplicate.

Complaint and Warning Management

Normal Scenario:

- A student or instructor files a complaint about a course, a peer, or another instructor.
- The system records the complaint with status OPEN and routes it to the registrar dashboard.
- The registrar reviews the complaint, optionally requests a dispute response from the target, and decides: dismiss, warn, or escalate.
- If a warning is issued, the target user's warningCount is incremented; crossing a threshold triggers consequences (account suspension, instructor review, or blacklisting).

Exceptional Scenarios:

- Complaint against non-existent target
    - Condition: Target user or course ID is invalid.
    - System response: Rejects the complaint and shows a validation error.
- Unauthorized resolution attempt
    - Condition: A user who is not the registrar tries to close or modify a complaint.
    - System response: Denies the action; displays "Only a registrar can resolve complaints."
- Warning threshold exceeded
    - Condition: User's warningCount reaches the institutional threshold.
    - System response: Account is suspended; the user is notified and can only access the system through an appeals workflow.

Apply for Graduation

Normal Scenario:

- A student opens the graduation application page once they meet the minimum credit requirement.
- The system auto-fills the student's transcript, major, GPA, and credit count.
- The student reviews the information and submits.
- The registrar verifies all graduation requirements against the student record and approves or rejects the application.
- On approval, the student's status is updated to GRADUATED and a graduation record is produced.

Exceptional Scenarios:

- Missing requirements
  - Condition: Student does not meet credit, GPA, or major-specific requirements.
  - System response: Submission is blocked or flagged for registrar review; missing requirements are listed to the student.
- Outstanding obligations
  - Condition: Student has outstanding complaints, incomplete grades, or unresolved holds.
  - System response: Application is put on hold; student is notified of the obstacles to clear before approval.
- Rejection by registrar
  - Condition: Registrar determines the student does not qualify despite meeting numeric thresholds.
  - System response: Application is rejected with a written reason; student may appeal or reapply.

Manage Courses and Semesters

Normal Scenario:

- The registrar opens the course-management interface.
- The registrar creates a new semester, then creates or edits courses for that semester, including title, credits, capacity, and schedule.
- The registrar assigns an instructor to each course from the list of active instructors.
- The registrar opens enrollment, which makes the semester visible to students for registration.

Exceptional Scenarios:

- Instructor unavailable
  - Condition: Assigned instructor's schedule conflicts with the course slot.
  - System response: Assignment is rejected; displays the conflict and prompts for another instructor or slot.
- Overlapping semester dates
  - Condition: A new semester's date range overlaps with an existing active semester.
  - System response: Creation is blocked; displays "Semester dates overlap with an existing active semester."
- Edit after enrollment opens
  - Condition: Registrar tries to reduce capacity below current enrollment.
  - System response: Edit is blocked; shows "Cannot set capacity below current enrolled count."

AI Chatbot Interaction

Normal Scenario:

- A user submits a question through the chat box.
- The system searches the local knowledge base for a confident match.
- If a match is found, the KB answer is displayed and the user is prompted to rate it.
- If no confident match is found, the system forwards the question (with any relevant KB context) to the LLM service and returns the generated answer.
- All chat sessions are logged with timestamp and user ID for auditing and for improving the knowledge base.

Exceptional Scenarios:

- Knowledge base or LLM unavailable
  - Condition: KB search or LLM call fails.
  - System response: Displays "Chat temporarily unavailable" or falls back to a canned response.
- Low-rated answer
  - Condition: A KB answer receives a rating of 1 (very low).
  - System response: Entry is flagged for registrar review; repeated low ratings cause the entry to be removed from the KB.
- Out-of-scope question

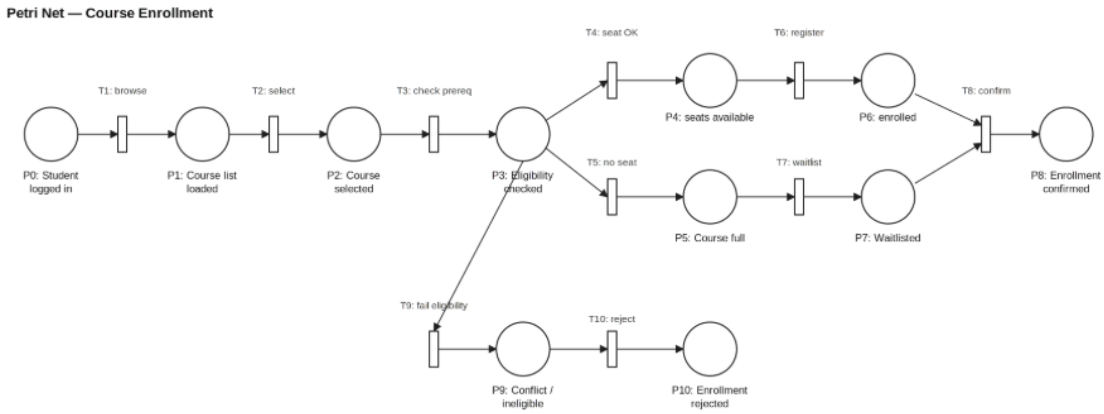
- Condition: User submits a question unrelated to CampusFlow (off-topic or abusive).
- System response: Chatbot declines to answer and optionally logs the interaction for moderation.

2.3 Petri Net

The phase 2 requirements call for three or more use cases to be represented as Petri nets instead of sequence or class diagrams. We selected the three use cases below because they combine a clear happy path, a branch on a condition, and well-defined exception flows, which are exactly the cases Petri nets model most clearly. In each diagram, circles are places (system states), rectangles are transitions (events), and directed edges connect them.

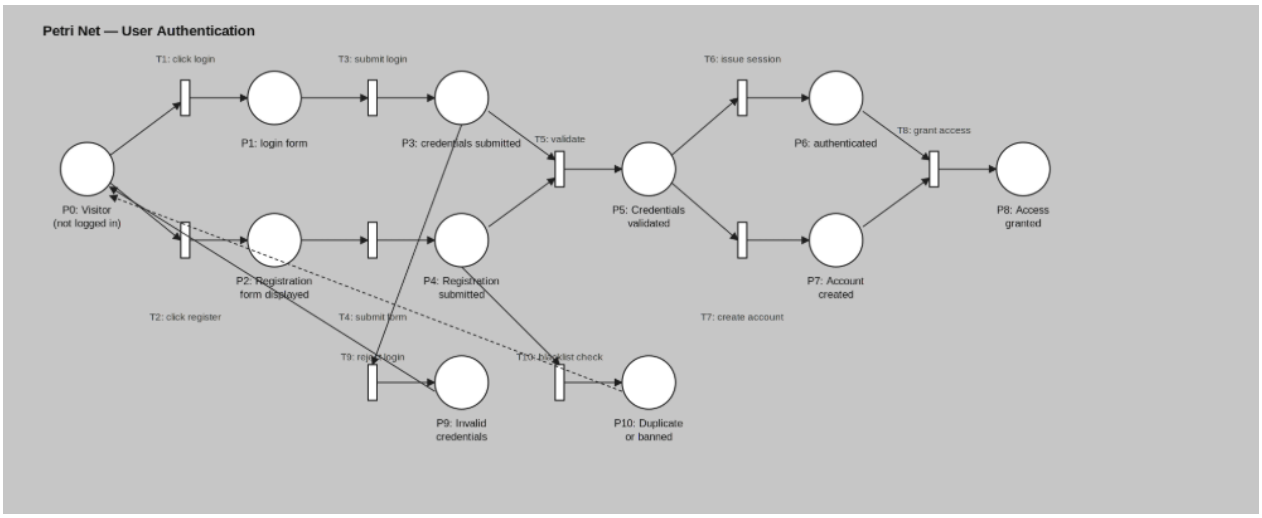
Petri Net 1 — Course Enrollment

A student enrolls in a course: the system loads the catalog, the student selects a course, prerequisites are checked, and the system branches on seat availability. If seats are open the student is enrolled; if not, the student is placed on the waitlist. The exception arm handles failed prerequisite or schedule-conflict checks and produces an enrollment-rejected state.



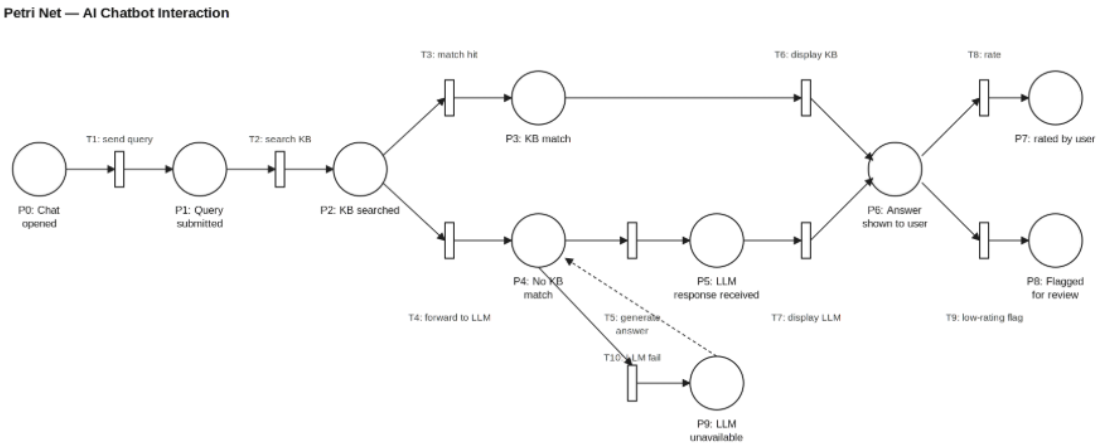
Petri Net 2 — User Authentication

The user begins as a Visitor and enters either the login branch or the registration branch. Both branches converge at a credential-validation transition. On success, either a session is issued (for login) or an account is created (for registration) and access is granted. The exception arms handle invalid credentials and duplicate or blacklisted registrations, returning the user to the Visitor state.



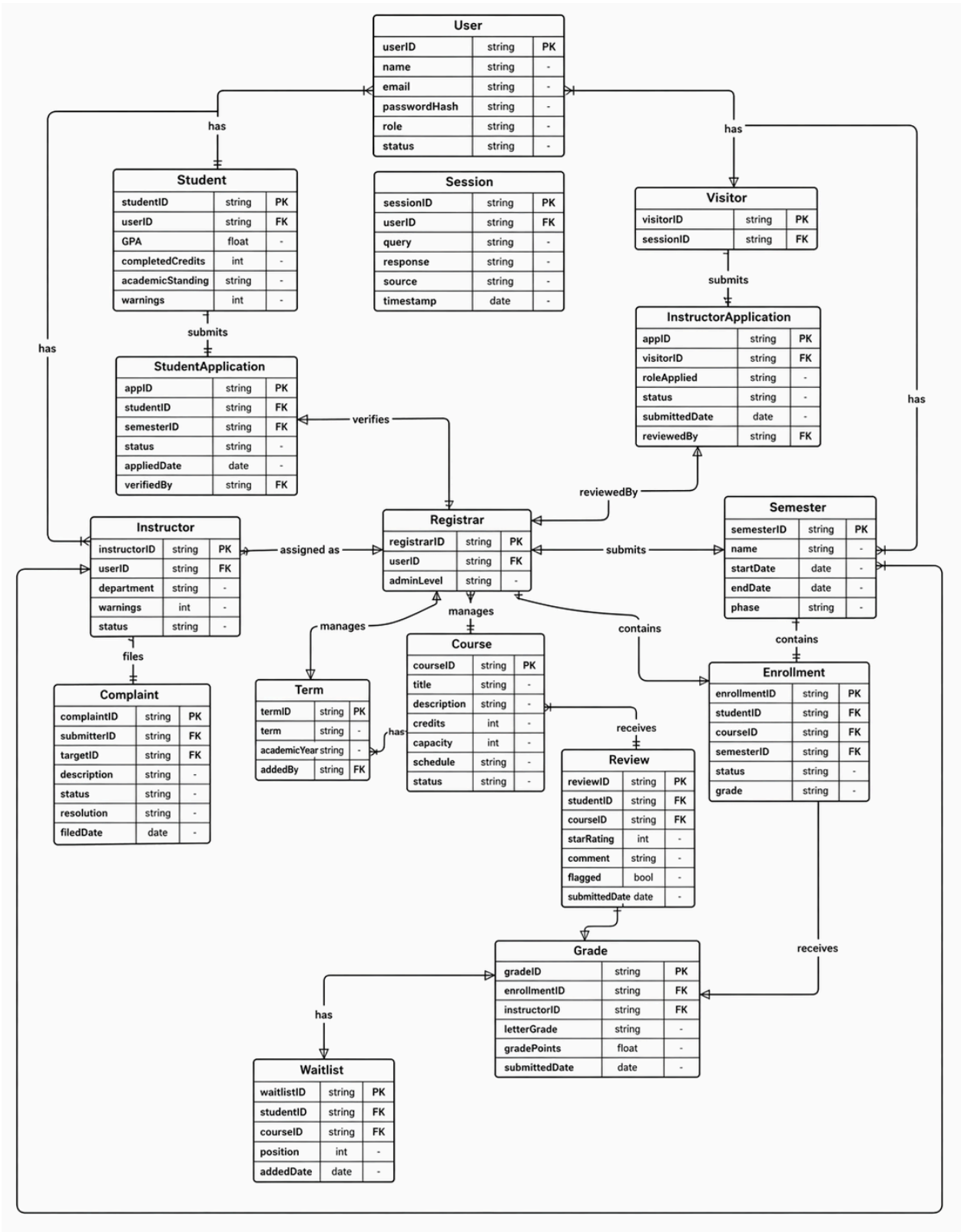
Petri Net 3 — AI Chatbot Interaction

The chatbot receives a query and searches the knowledge base. The net branches on whether a confident KB match was found: the KB path displays the stored answer and the LLM path generates a fresh one. Both paths converge at the answer-shown place, where the user can rate the answer (which may flag it for registrar review). The exception arm handles LLM unavailability by falling back to the no-match place so the system can retry or show a canned response.



3. Entity-Relationship (E-R) Diagram

3.1 Complete E-R Diagram of the System



3.2 Attributes and Keys for Each Class

User

Attribute	Type	Key
userID	string	PK
name	string	-
email	string	-
passwordHash	string	-
role	string	-
status	string	-

Student

Attribute	Type	Key
studentID	string	PK
userID	string	FK
GPA	float	-
completedCredits	int	-
academicStanding	string	-
warnings	int	-

Session

Attribute	Type	Key
sessionID	string	PK
userID	string	FK
query	string	-
response	string	-
source	string	-
timestamp	date	-

Visitor

Attribute	Type	Key
visitorID	string	PK
sessionID	string	FK

StudentApplication

Attribute	Type	Key
appID	string	PK
studentID	string	FK
semesterID	string	FK
status	string	-
appliedDate	date	-
verifiedBy	string	FK

InstructorApplication

Attribute	Type	Key
appID	string	PK
visitorID	string	FK
roleApplied	string	-
status	string	-
submittedDate	date	-
reviewedBy	string	FK

Registrar

Attribute	Type	Key
registrarID	string	PK
userID	string	FK
adminLevel	string	-

Instructor

Attribute	Type	Key
instructorID	string	PK
userID	string	FK
department	string	-
warnings	int	-
status	string	-

Complaint

Attribute	Type	Key
complaintID	string	PK
submitterID	string	FK
targetID	string	FK
description	string	-
status	string	-
resolution	string	-
filedDate	date	-

Term

Attribute	Type	Key
termID	string	PK
term	string	-
academicYear	string	-
addedBy	string	FK

Course

Attribute	Type	Key
courseID	string	PK
title	string	-
description	string	-
credits	int	-
capacity	int	-
schedule	string	-
status	string	-

Semester

Attribute	Type	Key
semesterID	string	PK
name	string	-
startDate	date	-
endDate	date	-
phase	string	-

Waitlist

Attribute	Type	Key
waitlistID	string	PK
studentID	string	FK
courseID	string	FK
position	int	-
addedDate	date	-

Review

Attribute	Type	Key
reviewID	string	PK
studentID	string	FK
starRating	int	-
comment	string	-
flagged	bool	-
submittedDate	date	-

Enrollment

Attribute	Type	Key
enrollmentID	string	PK
studentID	string	FK
courseID	string	FK
semesterID	string	FK
status	string	-
grade	string	-

Grade

Attribute	Type	Key
gradeID	string	PK
instructorID	string	FK
letterGrade	string	-
gradePoints	float	-
submittedDate	date	-

4. Detailed Design

AuthService:

```
class AuthService:
    """
    Manages identity verification, session lifecycle, and security.
    """

    def login(credentials: dict) -> dict:
        """
        Input: credential map containing 'email' and 'password_hash'.
        Output: session object of {user_id, role, status, auth_token}.

        Procedure:
        1. Find user by email.
        2. Check if password matches.
        3. Check if account is active.
        4. Generate auth token and return user info.
        """
```

```
        pass
    def logout(session_id: str) -> bool:
        """
        Input: session_id string representing the current active session.
        Output: Boolean indicating successful termination of the session.

        Procedure:
        1. Invalidate session token.
        2. Clear interface session state.
        3. Redirect to login page.
        """
        pass

    def register_visitor(visitor_data: dict) -> str:
        """
        Input: visitor_data dictionary containing 'name', 'email', and 'raw_password'.
        Output: user_id string of the newly created Visitor object.

        Procedure:
        1. Check if the email address is already registered.
        2. Hash the password.
        3. Create a new Visitor user record.
        4. Return the new User ID.
        """
        pass
```

ApplicationService:

```
class ApplicationService:
    """
    Manages key application services.
```

```
    """

    def submit_application(visitor_id: str, requested_role: str, academic_data: dict) -> str:
        """
        Input: visitor_id string, requested_role string ('Student' or 'Instructor'), academic_data
dictionary (containing GPA history, transcripts, or department requests).
        Output: app_id string of the newly generated application.
        Procedure:
        1. Check if visitor exists.
        2. Create pending application with data.
        3. Return application ID.
        """
        pass

    def evaluate_application(registrar_id: str, app_id: str, decision: str, notes: str) -> bool:
        """
        Input: registrar_id string, app_id string, decision string ('Approve' or 'Reject'), notes string
if decided to reject.
        Output: Boolean indicating transaction success.
        Procedure:
        1. Verify registrar privileges.
        2. If approved, create new Student or Instructor role and update status.
        3. If rejected, mark application as rejected.
        4. Save registrar notes and resolution date.
        """
        pass
```

CourseService:

```
class CourseService:
    """
```

```
    Handles the course lifecycle, including instructor assignment and course catalog management.
    """

    def create_course(registrar_id: str, course_data: dict) -> str:
        """
        Input: registrar_id string, course_data dictionary (containing Title, Description, Credits, Capacity, Schedule, Department).
        Output: course_id string.
        Procedure:
        1. Verify registrar privileges.
        2. Validate course details.
        3. Save new course to database.
        4. Return course ID.
        """
        pass

    def assign_instructor(registrar_id: str, course_id: str, instructor_id: str) -> bool:
        """
        Input: registrar_id string, course_id string, instructor_id string.
        Output: Boolean indicating successful assignment.
        Procedure:
        1. Verify registrar privileges.
        2. Check if instructor exists.
        3. Link instructor to course.
        """
        pass
    def update_course_availability(registrar_id: str, course_id: str, phase: str) -> bool:
        """
        Input: registrar_id string, course_id string, phase string ('Registration', 'Active', 'Grading', 'Closed').
```

```
        Output: Boolean indicating successful phase transition.
        Procedure:
        1. Verify registrar privileges.
        2. Update course phase (e.g., to 'Registration' or 'Active').
        """
        pass

    def browse_courses(filters: dict) -> list:
        """
        Input: filters dictionary (optional keys: Department, Instructor, Availability, Keyword).
        Output: List of detailed course dictionary objects.
        Procedure:
        1. Search courses using filters.
        2. Fetch ratings and enrollments.
        3. Return formatted list of courses.
        """
        pass
```

EnrollmentService:

```
class EnrollmentService:
    """
    Manages student course registration, timing conflicts, and waitlists.
    """

    def enroll_student(student_id: str, course_id: str) -> str:
        """
        Input: student_id string, course_id string.
        Output: Status string indicating the outcome ('Enrolled', 'Waitlisted', 'Failed_Conflict',
'Failed_Limit').
        Procedure:
        1. Check if course registration is open.
```

```
    2. Check if student has fewer than 4 courses.
    3. Check for schedule conflicts.
    4. If full, add to waitlist. Otherwise, enroll student.
    """
    pass

def drop_course(student_id: str, course_id: str) -> bool:
    """
    Input: student_id string, course_id string.
    Output: Boolean indicating successful removal.
    Procedure:
    1. Remove student's enrollment record.
    2. Check waitlist to fill the open seat.
    """
    pass

def check_time_conflict(existing_schedules: list, target_schedule: list) -> bool:
    """
    Input: existing_schedules list of time intervals, target_schedule list of time intervals.
    Output: Boolean True if a conflict in class times exists, else False.
    Procedure:
    1. Sort all class times.
    2. Check if any times overlap.
    3. Return true if there is a conflict.
    """
    pass

def process_waitlist(course_id: str) -> str:
    """
    Input: course_id string.
```

Output: student_id string of the user popped from the waitlist, or None if the waitlist is empty.

Procedure:

1. Find the oldest waitlist entry for the course.
2. Check if the student can still take the course.
3. Move student to enrolled status.

"""

pass

GradeService:

```
class GradeService:
    """
    Handles grade submission, cumulative GPA calculation, and academic standing rules enforcement.
    """

    def submit_grade(instructor_id: str, student_id: str, course_id: str, letter_grade: str) -> bool:
        """
        Input: instructor_id string, student_id string, course_id string, letter_grade string (e.g., 'A', 'B+', 'C').
        Output: Boolean indicating successful submission.
        Procedure:
        1. Verify course is in grading phase.
        2. Save the grade for the student.
        3. Update student's GPA.
        """
        pass

    def calculate_gpa(student_id: str) -> float:
        """
        Input: student_id string.
```

```
        Output: Float representing the updated cumulative GPA.
        Procedure:
        1. Get all completed course grades.
        2. Calculate average based on credits.
        3. Save new GPA.
        """
        pass

    def check_academic_standing(student_id: str) -> str:
        """
        Input: student_id string.
        Output: Standing status string ('Good', 'Warning', 'Suspension', 'Termination').
        Procedure:
        1. Check current GPA against minimum requirements.
        2. Issue warnings or suspend account if GPA is too low.
        """
        pass
```

ReviewService:

```
class ReviewService:
    """
    Manages the submission of course evaluations and performs content moderation.
    """

    def submit_review(student_id: str, course_id: str, rating: int, text: str) -> bool:
        """
        Input: student_id string, course_id string, rating integer (1-5), text string.
        Output: Boolean indicating successful submission.
        Procedure:
        1. Verify student completed the course.
        2. Check text for bad words.
```

```
        3. If bad words found, flag and hide review.
        4. Otherwise, save and update course rating.
        """
        pass

    def filter_inappropriate_content(text: str) -> bool:
        """
        Input: raw review text string.
        Output: Boolean True if inappropriate content is detected, False otherwise.
        Procedure:
        1. Compare text against banned words list.
        2. Return true if bad words are found.
        """
        pass
```

ComplaintService:

```
class ComplaintService:
    """
    Handles formal complaint submissions, tracking, and following disciplinary actions.
    """

    def submit_complaint(submitter_id: str, target_id: str, details: str) -> str:
        """
        Input: submitter_id string, target_id string, details string.
        Output: complaint_id string.
        Procedure:
        1. Verify users exist.
        2. Save complaint details as pending.
        3. Return complaint ID.
        """
        pass
```

```
    def process_complaint(registrar_id: str, complaint_id: str, action: str, resolution_notes: str) -> bool:
        """
        Input: registrar_id string, complaint_id string, action string ('Dismiss', 'Issue_Warning', 'Suspend'), resolution_notes string.
        Output: Boolean indicating successful processing.
        Procedure:
        1. Verify registrar privileges.
        2. Issue warning to target if needed.
        3. Mark complaint as resolved with notes.
        """
        pass

    def issue_warning(user_id: str, reason: str) -> bool:
        """
        Input: user_id string, reason string.
        Output: Boolean indicating successful application of the warning.
        Procedure:
        1. Add a warning to the user's record.
        2. Suspend user if they have too many warnings.
        """
        pass
```

GraduationService:

```
class GraduationService:
    """
    Manages graduation applications and verifies requirements for graduation.
    """

    def apply_for_graduation(student_id: str) -> str:
```

```
        """
        Input: student_id string.
        Output: Application status string.
        Procedure:
        1. Check if student passed at least 8 courses.
        2. If yes, create graduation application.
        """
        pass

    def verify_graduation(registrar_id: str, student_id: str) -> bool:
        """
        Input: registrar_id string, student_id string.
        Output: Boolean indicating successful verification.
        Procedure:
        1. Verify registrar privileges.
        2. Check student record manually.
        3. Approve application and mark user as Alumni.
        """
        pass
```

ChatService:

```
class ChatService:
    """
    Provides conversational AI assistance utilizing a local knowledge base.
    """

    def process_query(user_id: str, role: str, query: str, session_id: str) -> str:
        """
        Input: user_id string, role string, query string, session_id string.
        Output: AI Response string.
        Procedure:
```

```
        1. Search local knowledge base based on user role.
        2. If local answer is found, return it.
        3. If not, use external AI and add hallucination warning.
        4. Save chat history.
        """
        pass

    def retrieve_from_local_kb(query: str, scope_filter: dict) -> str:
        """
        Input: query string, scope_filter dictionary.
        Output: Extracted textual context string.
        Procedure:
        1. Convert query to vector.
        2. Find matching documents in local database.
        3. Return matching text.
        """
        pass
```

SessionService:

```
class SessionService:
    """
    Stores and retrieves chatbot history to maintain conversation ability.
    """

    def create_chat_session(user_id: str) -> str:
        """
        Input: user_id string.
        Output: new session_id string.
        Procedure:
        1. Create a unique session ID.
        2. Link it to the user.
```

3. Return session ID.

"""

pass

def log_interaction(session_id: str, query: str, response: str, source: str) -> bool:

"""

Input: session_id string, user query string, system response string, source string ('Local_KB' or 'External_LLM').

Output: Boolean indicating successful logging.

Procedure:

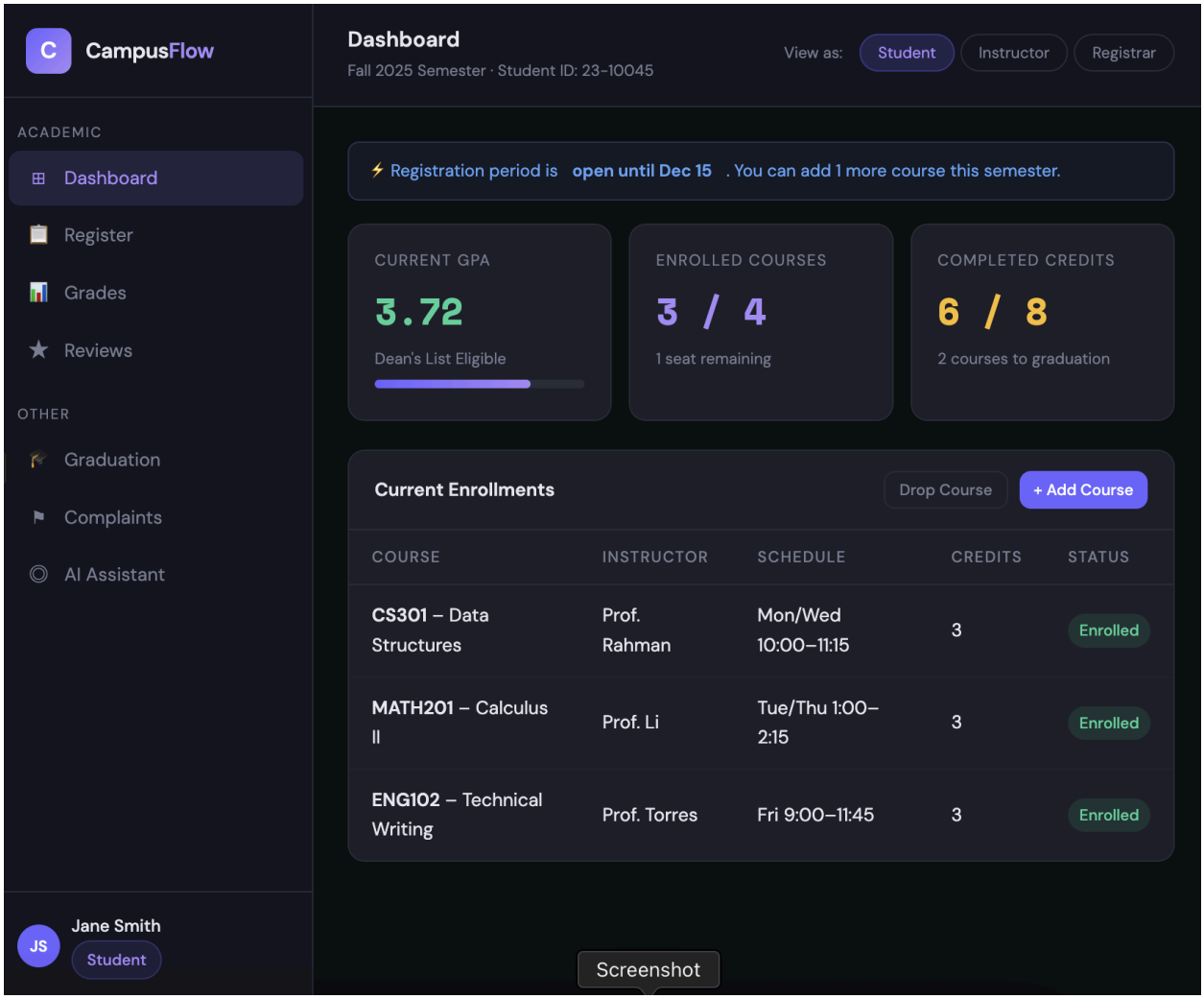
1. Save the query and response with a timestamp.

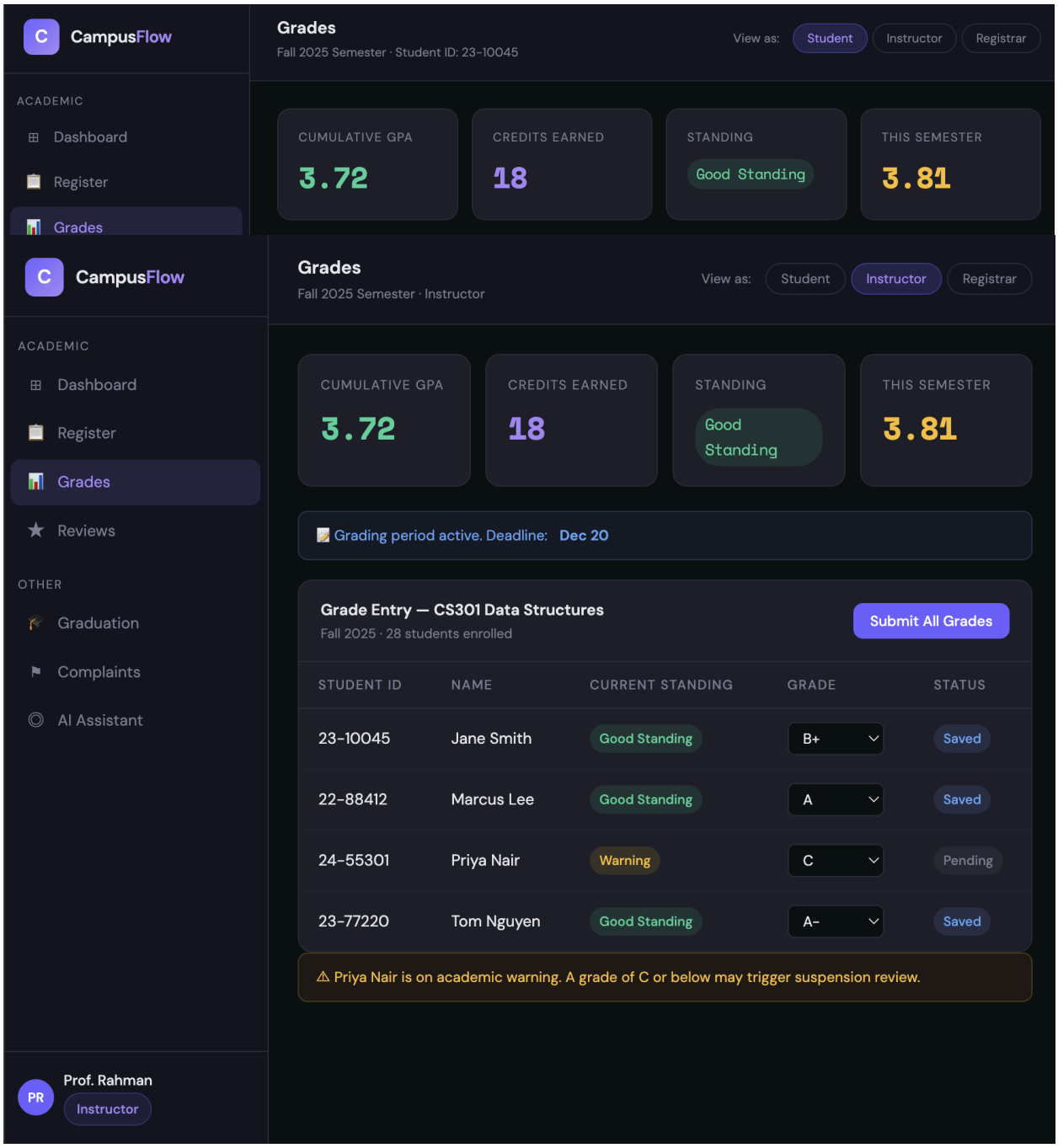
"""

pass

5. System Screens

5.1 Major GUI Screens

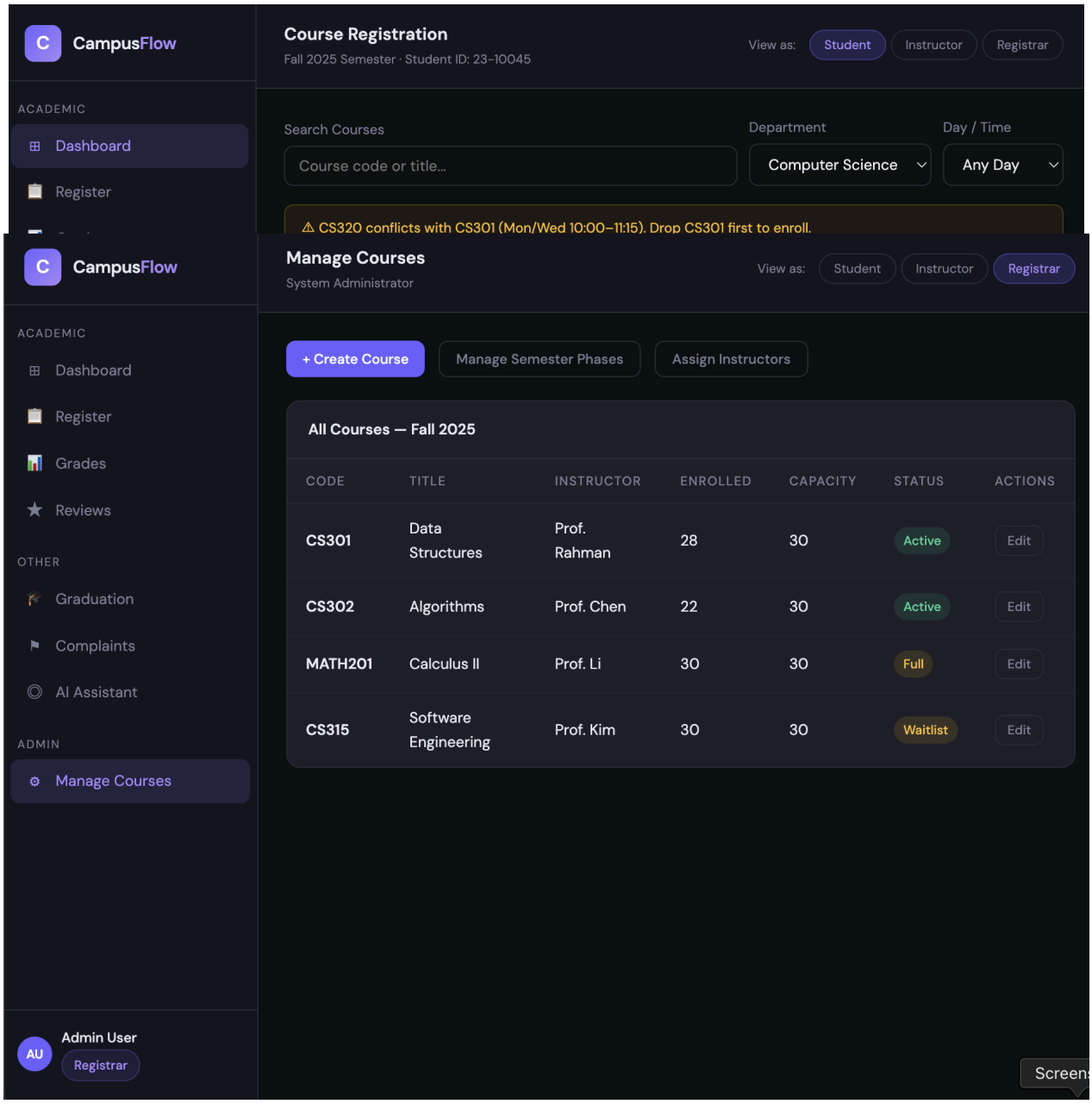




5.2 Sample Prototype of Selected Functionality

- The prototype of the software illustrates the core functionality of CampusFlow for

various user roles, namely, for students, instructors, and registrars. Thus, students are allowed to search for a particular course, enroll or drop out of courses, and track their progress in terms of GPA and credit completion. Instructors have the ability to access information about enrolled students and enter their grades. For their part, registrars deal with courses, semesters, graduation criteria, and student complaints.



6. Team Work Memos

Meeting Date: April 12

The group met to discuss the overall plan for the report and the direction of the project. The decision was made to switch from React to Streamlit because it allowed faster progress. Tausif and Arif began work on the menu interface, and everyone agreed on preparing the introduction, use cases, diagrams, and system screens well before the deadline.

Meeting Date: April. 12
A work division was assigned. Kapil took responsibility for the ER diagram. Ibsan handled the introduction, while Tausif worked on documenting all use cases and their scenarios. Arif agreed to create Petri-nets once the use cases were finalized. The group confirmed that everyone would later prepare pseudo-code for their respective methods.

Meeting Date: April. 14
Progress was reviewed. Kapil completed the menu pages, and started the cart functionality. Ibsan and Tausif identified several bugs during testing, including alignment issues and minor logic errors, which the team discussed and resolved together. Everyone kept contributing to the shared repository as changes were made.

Meeting Date: April. 16
The team revisited report requirements and confirmed that all sections were on track. With the deadline approaching, members planned to finish their assigned components and begin assembling the report. Concerns were mentioned about timing, diagram consistency, and the need to finalize use cases soon so Petri-nets and pseudo-code could be completed without delay.

7. Git Repository

Repository address (placeholder): https://github.com/Kapil-Tamang/CSC322-Project

End of Phase II: Design Report (draft).